Laboratorio · Entrada y salida en Java

Sistemas Distribuidos · 61TI · 615000735 · 4.º curso · Grado en Tecnologías para la Sociedad de la Información

ETSISI – UPM · Curso 2026-27 · Aula 3203 · martes 22 de septiembre de 2026 · Práctica 2: flujos de E/S y eco monohilo

Gustavo Hernández-Peñaloza y Javier Serrano (2024) · revisión de Carlos Castilla Ruiz y Guillermo Iglesias Hernández (2025)

Edición 2026-27 en castellano: David Díaz Pardo de Vera · david.diazp@upm.es · Despacho 4307

Basado en las transparencias 3-Input_Output (Input and output in Java): Hernández-Peñaloza y Serrano (2024), revisadas por Castilla e Iglesias (2025), según las portadas de las ediciones de 2024 y 2025, y las propiedades del fichero de 2025. En Moodle, «Materiales de cursos anteriores».

Clase de laboratorio · martes 22 de septiembre · 19:00–21:00

Primera parte de las transparencias; después, sockets y la práctica 2

Transparencias en castellano; cada término técnico lleva su forma inglesa entre paréntesis la primera vez que aparece, así: flujo de datos (ingl. *stream*). Los nombres de clases y de métodos, y las palabras reservadas de Java, se escriben tal cual. Glosario del curso: SD_Glosario_2627.

Cómo se organiza esta parte

Las transparencias van en dos partes, breves las dos, y después la práctica 2 en vuestros portátiles.

§	Sección	Minutos
1	Recursos y flujos de datos: qué es un flujo y cuáles son los cuatro pasos	4
2	Flujos de bytes con tipo: `DataInputStream`, `DataOutputStream`	10
3	Flujos de caracteres: `Reader`, `Writer` y las líneas	8
4	De los ficheros a la red: el puente a los sockets	1
	Transparencias de sockets, y después la práctica 2	el resto de la clase

En Moodle: el enunciado de la práctica 2 (ES y EN), este PDF, el de sockets y el glosario del curso (SD_Glosario_2627).

¿Qué tienen en común un fichero, el teclado y una conexión de red?

Cuatro minutos

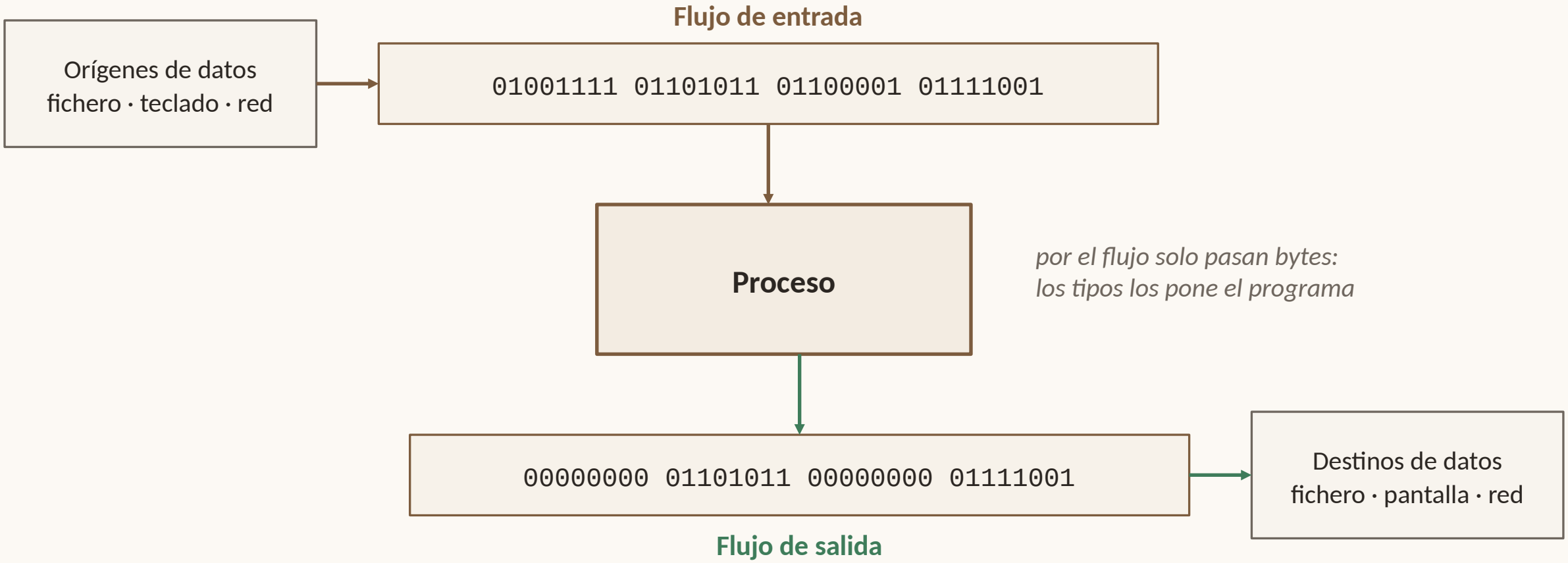
Recurso y flujo de datos

- **Recurso** (ingl. *resource*): origen o destino de datos binarios, con un nombre
- `/dir/fichero1` · `138.100.55.4:40` · el teclado · la pantalla
- **Flujo de datos** (ingl. *stream*): la tubería por la que esos bytes van y vienen
- De entrada (ingl. *input stream*) para leer · de salida (ingl. *output stream*) para escribir

Un recurso ofrece flujos; el programa solo ve flujos.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 2

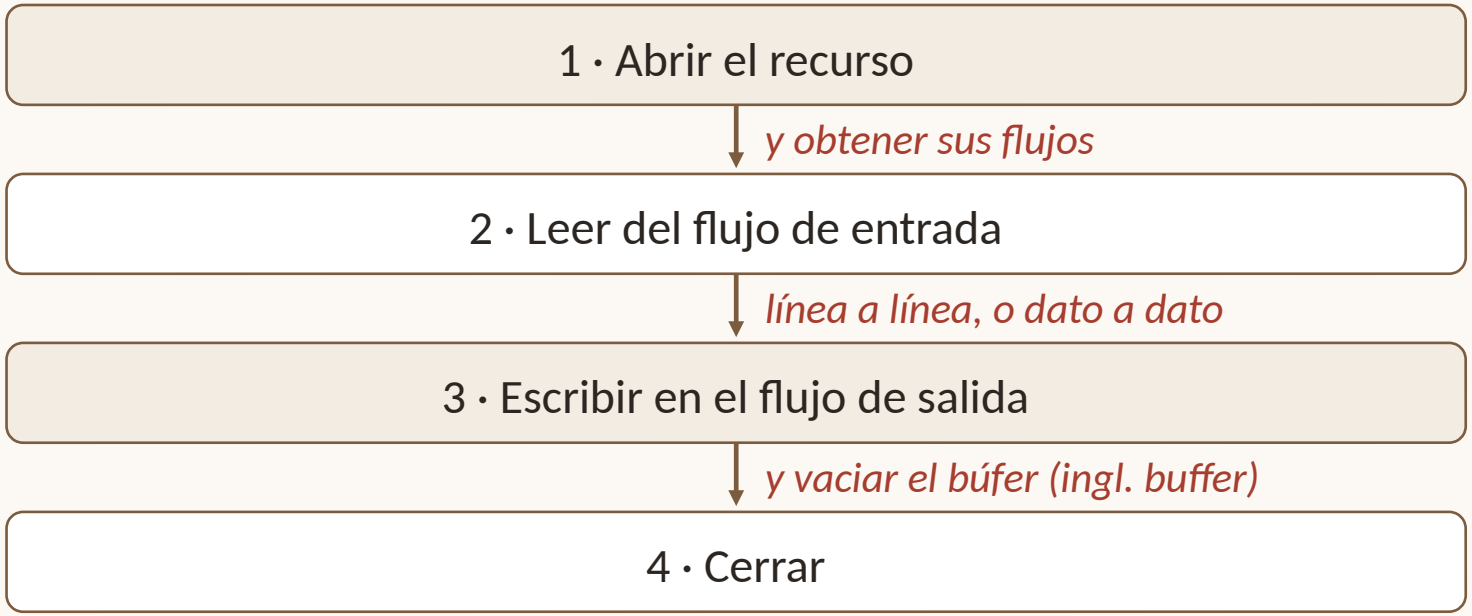
Un proceso, entre dos flujos



El proceso ve dos flujos de bytes. Los bytes no llevan etiqueta de tipo.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 3 (figura redibujada)

Los cuatro pasos, siempre los mismos



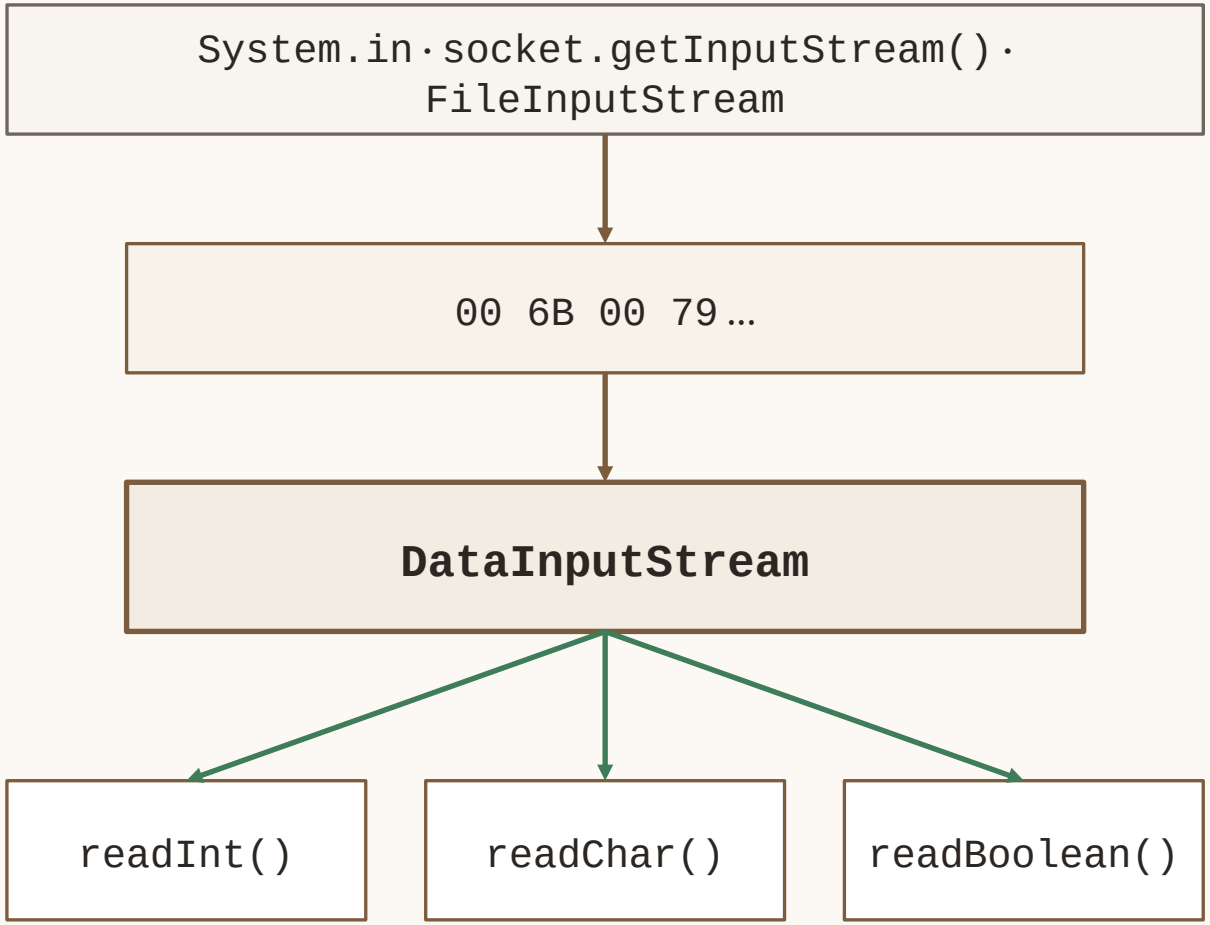
- 2 y 3 se repiten mientras haya datos
- Vale para un fichero, para el teclado y para un socket

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 4

Si por el flujo solo van bytes, ¿cómo viaja un `double`?

Diez minutos

`DataInputStream`: bytes que entran, tipos que salen



- Envuelve un flujo de bytes cualquiera
- Cada uno de estos métodos consume un número fijo de bytes
- `readInt()` → 4 · `readDouble()` → 8 · `readChar()` → 2 · `readBoolean()` → 1
- Quien escribe y quien lee acuerdan el orden y los tipos

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 5 (figura redibujada)

Los mismos cuatro bytes, tres lecturas distintas

Sobre el flujo `00 6B 00 79`, según el método que se llame:

Llamada	Bytes que consume	Resultado
`readInt()`	los 4	`7012473`
`readChar()` dos veces	2 + 2	`'k'` y `'y'`
`readBoolean()` cuatro veces	1 + 1 + 1 + 1	`false`, `true`, `false`, `true`

► Los bytes no cambian. Cambia lo que el programa dice que son.

Valores verificados con JDK 21.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 5–8 (valores corregidos y verificados)

`readUTF()`: primero la longitud, después el texto

`writeUTF("hola")` escribe seis bytes: `00 04` y después `68 6F 6C 61`.

- Los dos primeros bytes dicen cuántos bytes de texto vienen detrás
- `readUTF()` lee la longitud y luego esa cantidad exacta
- Sobre `00 6B 00 79`: longitud 107, solo quedan 2 bytes → `EOFException`
- Con `writeUTF` al otro lado funciona siempre: la longitud viaja con el texto

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 7 y 9 (comportamiento corregido y verificado con JDK 21)

Escribir tipos y volver a leerlos

Un `int`, un `double` y una cadena, escritos y recuperados en el mismo orden.

```
 1  DataOutputStream dos = new DataOutputStream(salida);
 2  dos.writeInt(42);
 3  dos.writeDouble(1.5);
 4  dos.writeUTF("hola");
 5  dos.flush();                    // sin esto, nada sale del búfer
 6
 7  DataInputStream dis = new DataInputStream(entrada);
 8  int    n = dis.readInt();       // 4 bytes
 9  double x = dis.readDouble();    // 8 bytes
10  String s = dis.readUTF();       // 2 de longitud + 4 de texto
```

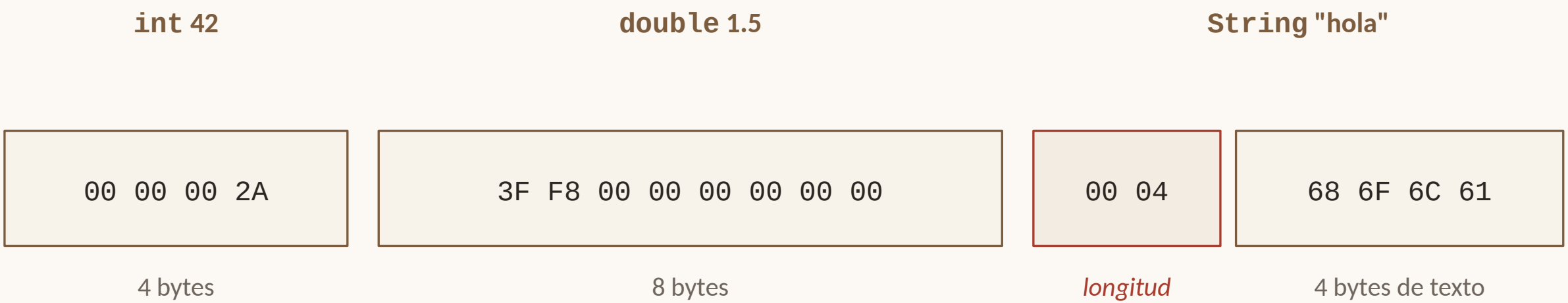
Salida:

```
n=42  x=1.5  s="hola"
18 bytes en total
```

El orden de lectura calca el orden de escritura. Si se intercambian dos lecturas, no avisa nadie.

Fuentes: Ejemplo nuevo de la edición 2026-27, compilado y ejecutado con JDK 21

Los 18 bytes que viajan



Big-endian («orden de red»): el byte más significativo primero, igual en cualquier máquina

La cadena es el único campo sin tamaño fijo: por eso lleva la longitud delante.

Fuentes: Figura nueva de la edición 2026-27; bytes obtenidos ejecutando el listado anterior con JDK 21

El final del flujo tiene dos caras

Protocolo	Cómo se manifiesta el final	Qué hace el código
De texto, por líneas	`readLine()` da `null`: ya no hay más líneas	sale del bucle
Binario, con tipos	`readInt()` lanza `EOFException`	captura la excepción y cierra

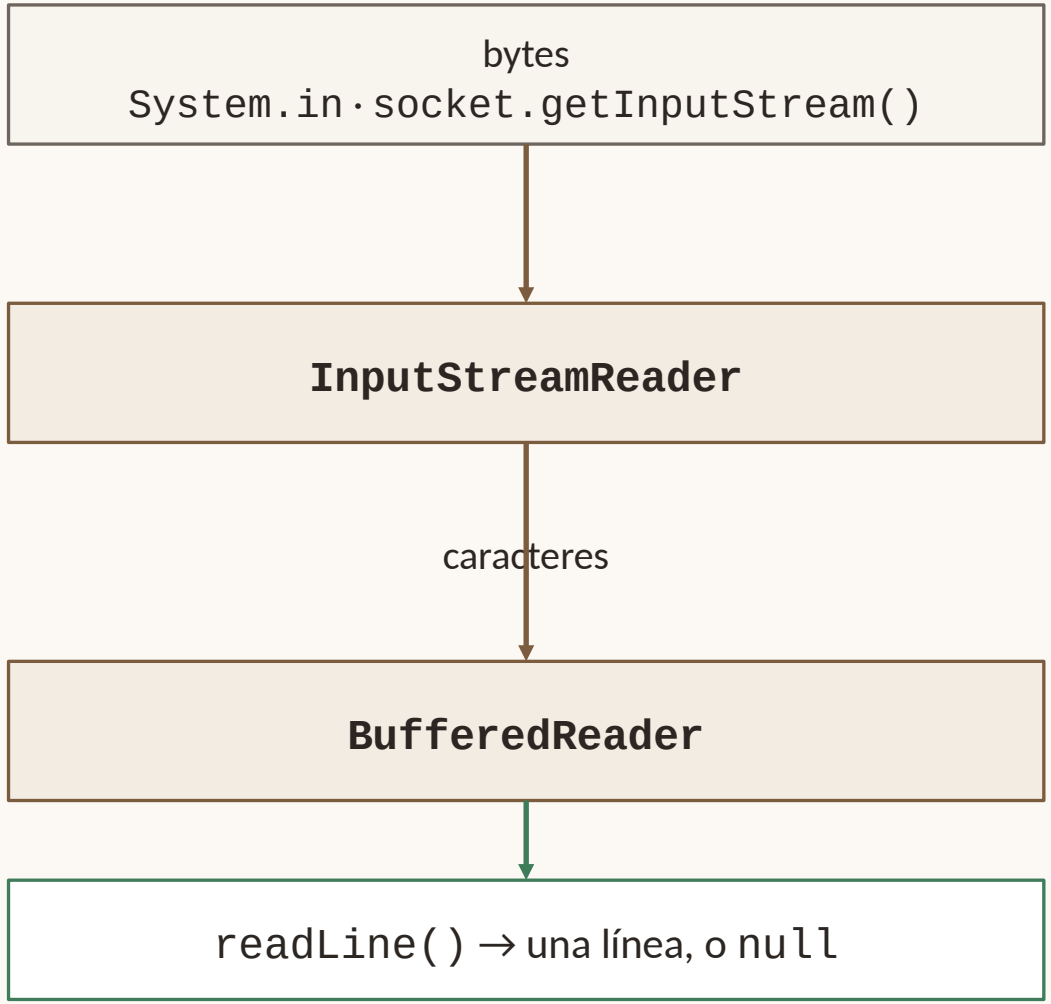
Las dos son el final normal de una conversación.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 13 y 18 (`null`); el caso binario es nuevo de la edición 2026-27

¿Y cuando lo que viaja son líneas de texto?

Ocho minutos

De bytes a líneas: la cadena de lectura



- `InputStreamReader`: bytes → caracteres, según una codificación (ingl. *encoding*)
- `BufferedReader`: caracteres → líneas
- Para un fichero de texto, `FileReader` sustituye el flujo de bytes y el `InputStreamReader`
- Una línea vacía es una cadena de longitud cero; `null` es el fin del flujo

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 11–12 (figura redibujada)

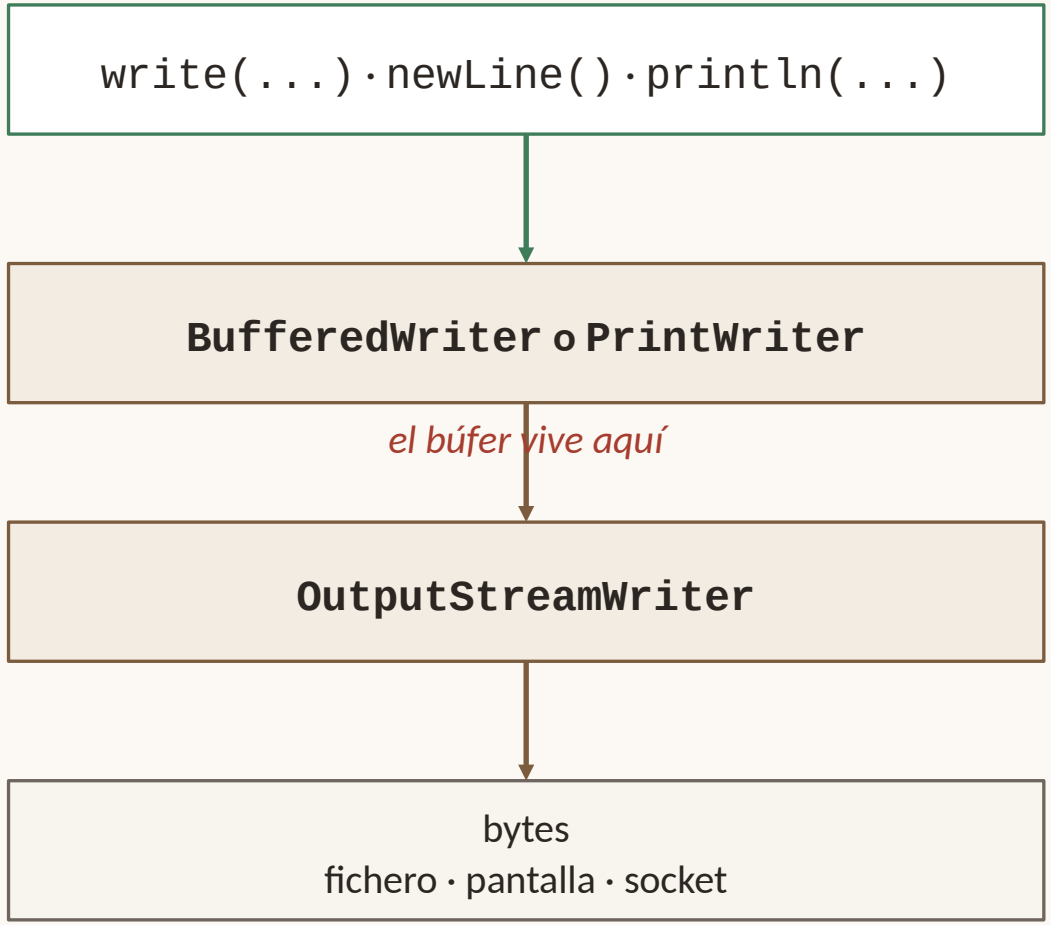
Leer del teclado hasta el fin de fichero

```
1  InputStreamReader lectorCaracteres = new InputStreamReader(System.in);
2  BufferedReader lectorLineas = new BufferedReader(lectorCaracteres);
3
4  String linea = lectorLineas.readLine();
5  while (linea != null) {              // null = fin de fichero
6      System.out.println(linea);
7      linea = lectorLineas.readLine();
8  }
9  System.out.println("fin de fichero");
10 lectorLineas.close();
```

Fin de fichero desde el teclado: Ctrl-D en Linux y macOS, Ctrl-Z y Enter en Windows.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 12–13 (traducido; `readline` corregido a `readLine`)

De líneas a bytes: la cadena de escritura



- La escritura es la lectura al revés: líneas, caracteres, bytes
- `FileWriter` hace los dos pasos de abajo para un fichero de texto
- `PrintWriter` añade `println`: lo que usa la práctica 2
- Nada llega al otro extremo hasta que el búfer se llena o se vacía

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 15–16 (figura redibujada)

Un `Writer` no escribe inmediatamente

Escribir «Hola» en un fichero y mirarlo antes de vaciar el búfer: 0 bytes.

- `OutputStreamWriter` y `FileWriter` acumulan en un búfer interno
- `flush()` lo vacía · `close()` lo vacía y cierra
- En la red: búfer sin vaciar → cliente esperando para siempre
- `new PrintWriter(flujo, true)`: autovaciado (ingl. *autoflush*), vacía en cada `println`

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 15 (afirmación corregida y medida con JDK 21)

Del teclado a un fichero

```
 1  BufferedReader teclado =
 2      new BufferedReader(new InputStreamReader(System.in));
 3  BufferedWriter fichero =
 4      new BufferedWriter(new FileWriter("teclado.txt"));
 5
 6  String linea = teclado.readLine();
 7  while (linea != null) {
 8      fichero.write(linea);
 9      fichero.newLine();
10      linea = teclado.readLine();
11  }
12  teclado.close();
13  fichero.close();                    // cierra y vacía el búfer
```

Quitar el último `close()` deja el fichero vacío. Es el mismo fallo que en la red.

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 18 (traducido y verificado con JDK 21)

Qué clase usar

Lo que se mueve	Para leer	Para escribir
Líneas de texto	`BufferedReader·readLine()`	`PrintWriter·println()`
Datos con tipo	`DataInputStream·readInt()`	`DataOutputStream·writeInt()`
Bytes en bruto	`InputStream·read()`	`OutputStream·write()`

Práctica 2 y práctica 3: la primera fila. Práctica 4: la segunda.

Fuentes: Tabla nueva de la edición 2026-27

Lo que hay que saber hacer

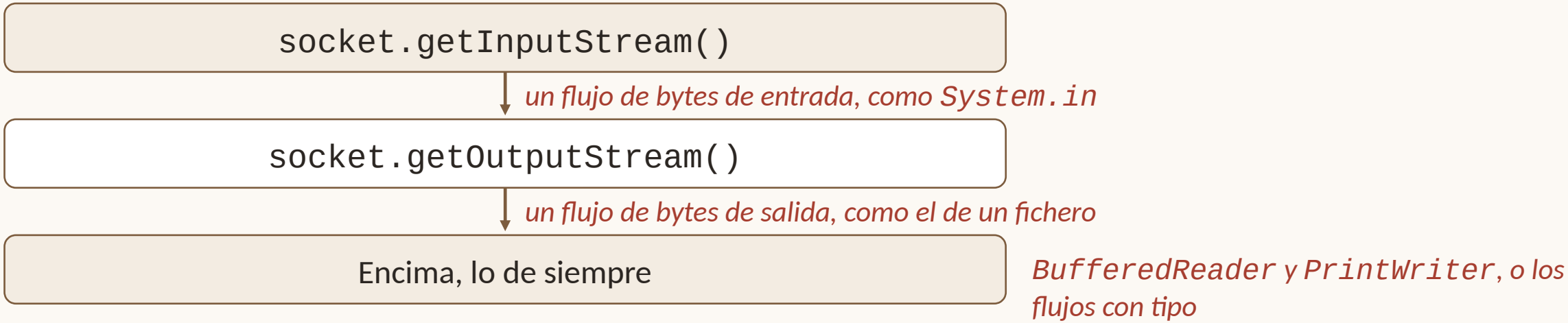
1. Montar la cadena de lectura de líneas sobre cualquier flujo de bytes
2. Escribir líneas con `PrintWriter` y autovaciado
3. Escribir y leer datos con tipo: `DataOutputStream` y `DataInputStream`
4. Decir cuántos bytes ocupa en el cable un `int`, un `double`, o una cadena con `writeUTF`
5. Reconocer el final del flujo: `null` en texto, `EOFException` en binario

Fuentes: Enunciado de la práctica 2 · Guía de aprendizaje 2026-27: PR-EX-1, «Java. TCP y Marshalling/Unmarshalling». En este curso, prácticas 0 a 5

¿Qué cambia cuando el recurso está en otra máquina?

Un minuto

Un socket da los mismos dos flujos



- Cambia la línea que abre el recurso; el resto del código es el de hoy
- El otro extremo puede desaparecer · sin vaciar el búfer, nada llega

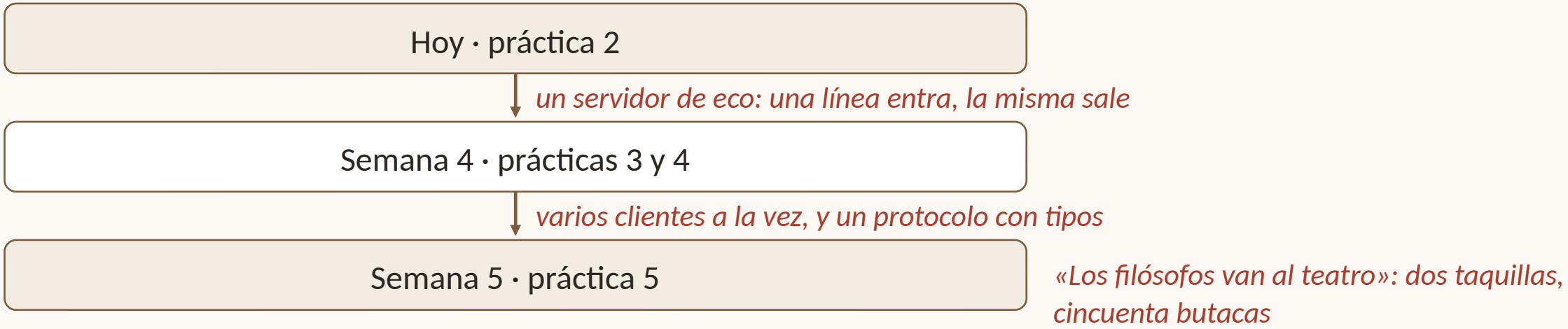
Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 4-Sockets, transp. 12

Para ampliar

Recurso	Dónde
Oracle, API de Java 21: `DataInputStream`	docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/DataInputStream.html
Oracle, The Java Tutorials: Basic I/O	docs.oracle.com/javase/tutorial/essential/io/
Baeldung, How to Read a File in Java	www.baeldung.com/reading-file-in-java
Enunciado de la práctica 2 y transparencias de sockets	Moodle, sección Laboratorio I

Fuentes: Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 3-Input_Output, transp. 19 (enlaces actualizados a Java 21)

Adónde lleva esto



- El mismo cliente y el mismo servidor, con más gente dentro

Fuentes: Hoja de ruta del laboratorio 2026-27 (Moodle)